



Acharya.

A vernacular AI tutor for Indian school children (Class 6–10),
built on **Brooks–Iyengar fault-tolerant consensus** across three
independent LLMs.

Product Requirements Document — v1.0

Document version

v1.0 — baseline for MVP

Status

MVP shipped to local demo · WhatsApp pipeline
pending Meta BSP onboarding

Date

2026 — pre-seed cycle

Repository

github.com/writetozealdefense-svg/acharya-mvp

CONTENTS

What's in this document

1	Executive summary
2	Product overview — one-page infographic
3	The opportunity
4	Product vision & principles
5	Target users & personas
6	Solution overview — system diagram
7	Functional requirements (P0 / P1 / P2)
8	The Brooks–Iyengar consensus algorithm
9	Technical architecture
10	UX flows
11	Privacy, security, & data handling
12	Success metrics
13	Roadmap & milestones
14	Risks & open questions
15	Glossary & appendix

SECTION 1

Executive summary

Acharya is a WhatsApp-native AI tutor for Indian school children (Class 6–10) that responds in vernacular languages and verifies every answer across three independent LLMs before sending it. The verification engine is a direct application of the **Brooks–Iyengar fault-tolerant consensus algorithm** — treating each LLM as a noisy, occasionally faulty sensor, and fusing their outputs in a way that tolerates a minority of bad signals.

The product targets the largest under-served education population on Earth: roughly **250 million Indian children in school**, the majority of whom study in a vernacular language and cannot fact-check a chatbot. Single-LLM tutors will hallucinate confidently in those languages, and neither the child nor the parent has a way to know. Acharya makes verification a structural property of the product, not a disclaimer.

What is built today

- End-to-end MVP demo on Next.js 14 with a WhatsApp-look chat at `localhost:3000`.
- Real Brooks–Iyengar consensus across **Claude Sonnet 4.5**, **GPT-4o-mini**, and **Gemini 2.0 Flash**, calling all three in parallel and using a fourth Claude call as the fact-agreement judge.
- Claude Sonnet vision for image-to-question extraction (no client-side OCR).
- Curriculum-grounded retrieval over 16 textbook excerpts in Kannada, Hindi, and English.
- Three pre-loaded demo scenarios including a hallucination-catch scenario showing what happens when models disagree.
- Parent dashboard with subject breakdown, concept tracker, and recent-questions log.

What unlocks scale

- WhatsApp Business API onboarding via a Meta BSP (AiSensy, Gupshup, or similar).
- Voice-note intake and reply, matching the rural-India usage pattern that already exists.
- Curriculum coverage expanded from a 16-excerpt seed corpus to full Class 6–10 across CBSE, NCERT, and the major State Boards.
- Family-account semantics (one parent, multiple children, per-child progress tracking).

The single most important thing to remember about Acharya

*Every other AI tutor on the market today ships answers wrapped in “AI-generated, may be inaccurate.” Acharya is the only one that ships with a verdict — **Verified** **Likely correct** or **Needs human review** — and an audit trail of why.*



Acharya.

A vernacular AI tutor for Indian school children. Built on **Brooks-Iyengar fault-tolerant consensus** across three independent LLMs — so when one model hallucinates, the other two catch it before a child sees it.

Class 6-10 · CBSE, NCERT, State Boards

Kannada · हिंदी · English · more languages on the roadmap

WhatsApp-native — no app to install

3 LLMs in parallel · Anthropic · OpenAI · Google

WHY ACHARYA EXISTS

The hallucination problem, in the country least equipped to catch it

The problem with single-LLM tutors

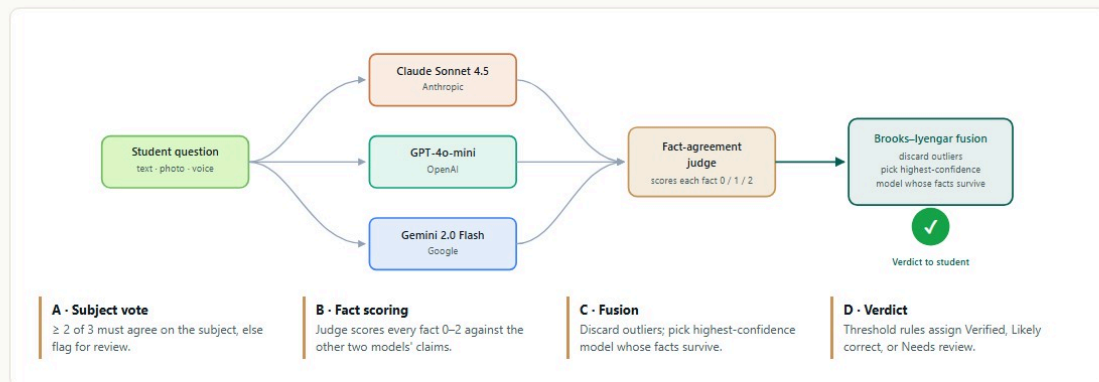
- One model, one voice, one chance to be wrong — confidently.
- A Class 7 child cannot fact-check a chatbot. Neither can their parent.
- Most Indian school children study in vernacular languages where models are weaker and hallucinations are more frequent.
- "AI-generated, may be inaccurate" disclaimers don't fix the lesson.

What Acharya does differently

- Every question runs through **three independent LLMs in parallel**.
- A fourth judge model scores how much each model's claims agree with the others'.
- Brooks-Iyengar fault-tolerant fusion discards outliers and picks the answer that the surviving consensus supports.
- Every reply ships with a verdict — **Verified**, **Likely correct**, or **Needs human review** — so parents see exactly how confident the system is.

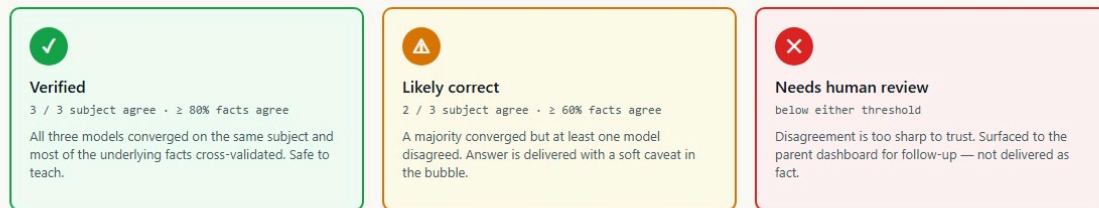
HOW IT WORKS

One question. Three witnesses. One verifiable answer.



EVERY ANSWER SHIPS WITH A VERDICT

The parent never has to guess how confident the AI is.



HOW A CHILD ACTUALLY USES IT

Three ways in. One WhatsApp thread out.



THE PEDIGREE

Why Brooks-Iyengar?

The Brooks-Iyengar algorithm, introduced by **Dr. S. S. Iyengar** and Richard Brooks, was originally a fault-tolerant consensus protocol for distributed sensor networks — fusing readings from multiple sensors when any minority of them might be faulty.

We treat each LLM as a sensor. Each one observes the same input — the student's question — and emits a noisy, occasionally faulty reading. The Brooks-Iyengar fusion step is structurally identical: tolerate a minority of faulty signals, fuse the rest into a single defensible output.

"The product is, structurally, a textbook application of the algorithm to LLM outputs treated as faulty sensors — the agreement panel is the same kind of graph distributed-systems researchers have been reading for two decades."

1996

Brooks-Iyengar published

3

LLMs treated as independent sensors

2026

Applied to vernacular education

Production-grade infrastructure, day one.

Anthropic

Claude Sonnet 4.5 · vision · judge

OpenAI

GPT-4o-mini · independent vote

Google

Gemini 2.0 Flash · independent vote

Meta

WhatsApp Business API (production target)

NCERT · CBSE · State Boards

Curriculum-grounded retrieval

Next.js + TypeScript

Vercel-ready demo + API routes

KaTeX

First-class math rendering

Brooks-Iyengar

The algorithm at the heart of it all

~7s

end-to-end latency from
question to verified
answer

3 + 1

model calls per question
— three answerers + one
judge

15s

per-model timeout · failed
models become non-
votes

0

data retained for child
users — no training, no
resale

Acharya · MVP demo — local Next.js simulation of the WhatsApp experience, running real LLM consensus against live APIs. Production target: WhatsApp via Meta Business API.

github.com/writetozealdefense-svg/acharya-mvp

Acharya · PRD v1.0

SECTION 3

The opportunity

Market sizing

India has approximately **250 million school children in Class 1–12** — the world's largest school-going cohort. Of these, roughly **120 million are in Class 6–10**, the age band where homework help becomes the dominant educational ask outside the classroom. A super-majority study in a vernacular medium of instruction (Hindi, Kannada, Tamil, Telugu, Marathi, Bengali, etc.). The premium English-medium private-tutor market is well-served; the vernacular medium Class 6–10 market is not.

Why now

- **WhatsApp penetration.** 500M+ Indian users; effectively the primary compute-and-communication surface for the family unit. No app to install, no UX to teach.
- **Frontier LLMs are now multilingual.** Claude, GPT, and Gemini all produce defensible Class 6–10 answers in Hindi and Kannada today — defensible *most of the time*. The remaining tail of hallucinations is the problem Acharya solves.
- **Per-question economics work.** Three LLM calls + one judge call costs in the order of cents per question at current pricing. Sustainable at parent-pays prices well below an in-person tutor.

The problem with what exists today

Single-LLM tutors

One model, one voice, one chance to be wrong. The child has no way to detect confident-sounding hallucinations — especially in a vernacular language where model accuracy is meaningfully lower than English.

Edtech apps

Require app installation, English UI, and a paid subscription. Effectively gated to urban English-medium families. Vernacular-medium parents in tier 2–4 cities are excluded from the design.

WhatsApp tutor groups

Manual, slow, and dependent on the goodwill of a teacher who is already overloaded. No verifiability, no audit trail, no progress tracking.

Generic ChatGPT-style apps

Not designed for children. No curriculum grounding. No parent visibility into what the child is asking or what verdict the AI gave. Disclaimers in legalese, not in actionable form.

Acharya's wedge

A WhatsApp-native vernacular tutor whose *verification* — not its answers — is the differentiator. The Brooks–Iyengar consensus engine is a defensible academic foundation that is also a memorable product story.

SECTION 4

Product vision & principles

Vision statement

Every Indian school child should have access to a tutor that speaks their language, sees their textbook, and admits when it might be wrong. Acharya is that tutor, on the one app every Indian family already uses.

Operating principles

1. Verification is structural, not cosmetic.

Every answer carries a verdict computed from real model agreement, not a static disclaimer. The verdict is visible in the chat bubble — not buried in settings.

2. Vernacular-first, English-second.

The product detects the child's language from the first message and replies in kind. Switching languages is a one-tap pill, not a settings menu.

3. The intake is whatever the child is comfortable with.

Type, snap a textbook page, or send a voice note. Acharya does the work of converting these into a question, not the child.

4. The parent is a first-class user.

The parent dashboard is a real product surface, not a marketing afterthought. Parents see what their child is asking, which subjects are getting Verified answers, and which topics are being flagged for review.

5. Honest failure beats confident hallucination.

If the consensus engine cannot reach a Verified or Likely correct verdict, the child sees a Needs review state and the question goes to the parent dashboard for follow-up. We do not paper over disagreement.

6. Built on giants' shoulders.

The product is a thin, opinionated application of well-established work — Brooks–Iyengar consensus, frontier LLMs, WhatsApp's distribution. We add taste and judgment, not new model research.

Explicit non-goals (v1.0)

- We are not building our own LLM, our own messaging app, or our own curriculum.
- We are not targeting Class 11–12 board prep or competitive-exam tutoring in v1.
- We are not building a parent paywall or per-question billing in v1; pricing is TBD and gated on retention data.
- We are not localising into the Devanagari numeral system unless user research shows a strong preference; we use Hindu-Arabic numerals which the textbooks use.

- We are not adding a teacher-facing dashboard in v1.

SECTION 5

Target users & personas

Primary persona — Aanya, Class 7, Hubballi

Profile. 12 years old, studies in a Karnataka State Board school, medium of instruction is Kannada. English is a second-language subject she finds difficult. She has access to her father's smartphone for an hour after dinner most weekdays.

Goal. Get help with that day's homework before her father loses patience or her mother needs the phone.

Trigger. A specific homework problem she does not understand.

Failure mode without Acharya. She copies an answer from a friend's notebook, or skips the question, or asks her father who is not a subject teacher. Conceptual gaps compound silently.

What success looks like. She types or photographs a question, receives a Kannada explanation in 8–10 seconds with a Verified badge, follows up with one or two clarifying questions, finishes the problem in under 4 minutes, and goes to bed.

Secondary persona — Vinod, Aanya's father

Profile. 42 years old, accountant, active WhatsApp user, completed Class 12 himself but has not done Class 7 science in 25 years. Has two children.

Goal. Know what his daughter is studying and whether she's understanding it — without having to learn the material himself.

What success looks like. Once a week he opens the parent dashboard, sees a green dot next to "Photosynthesis" and "Linear equations", an amber dot next to "Acids and bases", and a one-line summary like "12 questions this week, 9 Verified, 3 Likely correct, 0 review needed." He tells his daughter she's doing well in math and asks her to slow down on chemistry.

Tertiary persona — the school teacher (out-of-scope for v1)

Teachers are an obvious adjacent persona — they could use the same engine to prepare lessons, generate practice problems, and review classroom-wide question patterns. Explicitly out-of-scope for v1 to keep the surface area focused. Phase 3 on the roadmap.

SECTION 6

Solution overview

The end-to-end loop, from a child's question to a verified answer, is shown below. It runs in roughly 7 seconds today against live model APIs. Every box on the diagram is a real piece of code in the MVP repository.

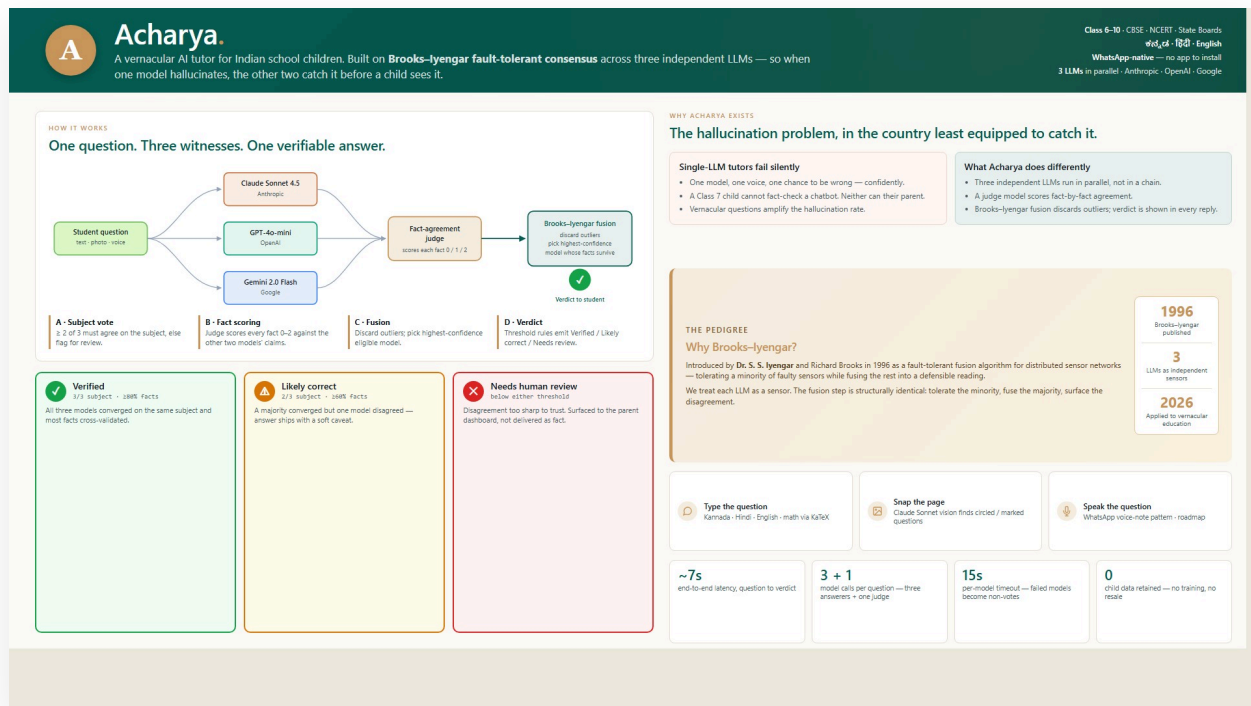


Figure 1. The Acharya consensus loop. The student question fans out to three independent LLMs in parallel, then a Claude judge call scores fact-by-fact agreement, then Brooks-Iyengar fusion picks the answer from the highest-confidence eligible model and assigns one of three verdicts based on threshold rules.

Why three models, not two or five

Two would be a tie when they disagree. Five would push per-question latency and cost past usable. Three is the smallest number that admits a meaningful majority vote — and three is exactly what Brooks-Iyengar fault-tolerant consensus assumes when tolerating a single faulty sensor (the canonical $3f+1$ setup with $f=1$ fault).

Why three different vendors

Independence of the votes is everything. Three calls to Claude would correlate. Three calls to OpenAI would correlate. Pulling Anthropic, OpenAI, and Google means three different training corpora, three different fine-tuning regimes, and three different RLHF policies — the closest you can get to true sensor independence in this domain.

SECTION 7

Functional requirements

Requirements are tagged **P0** (must ship for v1.0), **P1** (next quarter), **P2** (later, gated on traction).

FR-1 Conversational tutoring

ID	Requirement	Priority
FR-1.1	The system accepts a free-form question in Kannada, Hindi, or English.	P0
FR-1.2	The system detects the language from the question and replies in the same language.	P0
FR-1.3	Answers are 3–5 sentences for concept questions, step-by-step for math/science problems.	P0
FR-1.4	Math notation renders correctly via KaTeX in the chat bubble.	P0
FR-1.5	The user can type follow-up questions in the same thread; the engine treats each as an independent verification round (no cross-question caching in v1).	P0
FR-1.6	Add Tamil, Telugu, Marathi, Bengali, Gujarati to the language toggle.	P1

FR-2 Image intake

ID	Requirement	Priority
FR-2.1	The user can attach a PNG/JPEG photo of a textbook page.	P0
FR-2.2	Claude Sonnet vision extracts the most prominent question from the page (preferring circled/marked questions if present).	P0
FR-2.3	If no clear question is identified, the user is prompted to type the question while the image is preserved as context.	P0
FR-2.4	Voice-note intake (Whisper or equivalent) for questions and replies.	P1

FR-3 Verification engine

ID	Requirement	Priority
FR-3.1	Every answer goes through Brooks–Iyengar consensus across exactly three LLMs called in parallel: Claude Sonnet, GPT-4o-mini, Gemini 2.0 Flash.	P0
FR-3.2	Each model returns structured JSON with <code>answer</code> , <code>key_facts</code> (3–6 atomic claims), <code>confidence</code> (0–1), and <code>subject</code> .	P0
FR-3.3	A fourth Claude call acts as the fact-agreement judge, scoring each fact 0/1/2 against the other two models' fact lists.	P0

FR-3.4	Per-model timeout is 15 seconds; failed models become non-votes and the engine continues with the survivors.	P0
FR-3.5	The verdict is one of Verified / Likely correct / Needs human review per the threshold rules in Section 8.	P0
FR-3.6	The chat header has a "Show consensus reasoning" toggle that expands a panel under each assistant message showing all three model votes side-by-side.	P0
FR-3.7	Cross-question caching: identical curriculum questions reuse a recent verdict to reduce cost.	P1
FR-3.8	Adaptive sampling: skip the third model when the first two agree at high confidence on a low-stakes question.	P2

FR-4 Curriculum grounding

ID	Requirement	Priority
FR-4.1	A keyword-overlap retrieval pass selects up to 2 relevant excerpts from a curated textbook corpus for every question.	P0
FR-4.2	Retrieved excerpts are injected into every model's prompt under a <code><curriculum_context></code> tag with grounding instructions.	P0
FR-4.3	The seed corpus covers Class 7–8 across biology, chemistry, physics, math, history, geography (16 excerpts in v1).	P0
FR-4.4	Expand corpus to full Class 6–10 across CBSE, NCERT, Karnataka, Tamil Nadu, Maharashtra, West Bengal boards.	P1
FR-4.5	Replace keyword retrieval with embedding-based semantic retrieval once corpus exceeds ~500 excerpts.	P1

FR-5 Parent dashboard

ID	Requirement	Priority
FR-5.1	Summary cards: total questions this week, # Verified, # Needs review, average verdict score.	P0
FR-5.2	Subject breakdown as a donut chart with counts.	P0
FR-5.3	Concept tracker: green/amber/red dots per topic inferred from verdict patterns.	P0
FR-5.4	Recent questions list (last 20) with verdict, subject, language, timestamp.	P0
FR-5.5	Per-child accounts within a single parent account; switch child via a dropdown.	P1
FR-5.6	Weekly summary push to the parent's WhatsApp (one message, opt-in).	P1

FR-6 WhatsApp delivery (production target)

ID	Requirement	Priority
FR-6.1	Inbound webhook from Meta WhatsApp Business API processes text, image, and (eventually) voice messages.	P0
FR-6.2	The same <code>/api/ask</code> and <code>/api/extract-question</code> contracts power both the local demo and the WhatsApp webhook handler.	P0
FR-6.3	Outbound replies use WhatsApp's standard message envelope with the verdict prepended as an emoji + label.	P0

FR-6.4	BSP integration via AiSensy or Gupshup pending Meta approval.	P0
--------	---	----

FR-7 Demo & presentation

ID	Requirement	Priority
FR-7.1	A local demo server convincingly simulates the WhatsApp UI in the browser at localhost:3000 .	P0
FR-7.2	Three pre-loaded scenarios available from the chat header: Kannada concept question, Hindi algebra image, and the hallucination-catch demo.	P0
FR-7.3	The hallucination scenario uses a hardcoded model-vote injection (clearly labelled in source) to stage a disagreement reliably.	P0

SECTION 8

The Brooks–Iyengar consensus algorithm

The original Brooks–Iyengar algorithm (Brooks & Iyengar, 1996) was a fault-tolerant data fusion procedure for distributed sensor networks. Given N sensors of which up to f may be Byzantine-faulty, it produces a single fused output whose accuracy is bounded even in the presence of arbitrary failures, by **discarding outliers and weighing the remaining sensor outputs by overlap**.

Acharya treats each LLM as a sensor. The input — a tutoring question — is the same physical phenomenon being observed. Each LLM returns a noisy, occasionally faulty observation: an answer plus a list of factual claims it depends on. The fusion problem becomes: combine three observations into one trustworthy answer in a way that is robust to one model being wrong.

The four-step pipeline

Step A — Subject vote

Each model classifies the question into one of eight subjects (physics, chemistry, biology, math, history, geography, language, other). At least 2 of 3 must agree on the subject; if they don't, the question is flagged for human review immediately and no answer is delivered.

Step B — Fact-agreement scoring

Each model emits 3–6 atomic factual claims supporting its answer, in English so they can be compared cross-language. A fourth Claude call serves as the judge: for every claim made by every model, the judge inspects whether the other two models' claim lists contain agreeing assertions (semantic match, not string match). This yields a score in $\{0, 1, 2\}$ per claim — the number of other models that agreed.

Step C — Fault-tolerant fusion

Per-model statistics are computed: total facts, surviving facts (score ≥ 1), average score. *Eligible* models are those whose subject matches the majority subject, whose surviving fact set is non-empty, and whose average score is ≥ 1 . Among eligible models, the highest-confidence one is chosen as the answer source. If no model is eligible, the engine falls back to the highest-confidence ok-model and the verdict is forced to Needs review.

Step D — Verdict assignment

Verdict	Threshold
Verified	3 of 3 models agreed on subject AND $\geq 80\%$ of facts have score ≥ 1 AND eligible set is non-empty.
Likely correct	2 of 3 models agreed on subject AND $\geq 60\%$ of facts have score ≥ 1 AND eligible set is non-empty.
Needs human review	Below either of the above thresholds, OR the eligible set is empty.

Why this maps cleanly to the original algorithm

Brooks–Iyengar's contribution was the *geometric overlap* step — sensors whose intervals overlap with at least $N-f$ others contribute to the fused output, others are discarded. Our fact-agreement scoring is the LLM analogue: claims are intervals in semantic space, and the judge measures pairwise overlap. The "discard outliers, fuse the rest" operation is the same operation, lifted from real-valued sensor readings to natural-language claims.

SECTION 9

Technical architecture

MVP stack (today)

- **Frontend & API:** Next.js 14 App Router with TypeScript, Tailwind CSS, React 18.
- **Model SDKs:** `@anthropic-ai/sdk`, `openai`, `@google/generative-ai`.
- **Image rendering:** Claude Sonnet vision (no client-side OCR, no Tesseract).
- **Math:** `react-markdown` + `remark-math` + `rehype-katex`.
- **State:** React `useState` + `localStorage`. No database in the demo.
- **Icons:** `lucide-react`.

Production stack (target)

- **Messaging:** WhatsApp Business API via a Meta-approved BSP (AiSensy or Gupshup, TBD).
- **Webhook handler:** Same Next.js API routes deployed on Vercel or AWS.
- **Persistence:** Postgres (Neon or Supabase) for accounts, conversations, verdicts. Object storage (S3 or R2) for media.
- **Vector store (FR-4.5):** pgvector or Pinecone, only when curriculum corpus exceeds ~500 documents.
- **Observability:** Per-call latency, cost, verdict distribution, model failure rates. Tooling TBD; native Vercel + a thin custom dashboard initially.

Repository structure

Path	Role
<code>app/page.tsx</code>	WhatsApp-style chat UI (client)
<code>app/dashboard/page.tsx</code>	Parent dashboard (client)
<code>app/api/ask/route.ts</code>	Question → verified answer endpoint
<code>app/api/extract-question/route.ts</code>	Image → extracted question endpoint
<code>lib/consensus.ts</code>	Brooks-Iyengar algorithm — the heart of the product
<code>lib/models.ts</code>	Three SDK adapters with a unified <code>ModelResponse</code> interface
<code>lib/retrieval.ts</code>	Keyword retrieval over the curriculum corpus
<code>lib/prompts.ts</code>	System prompts for tutor, consensus, judge, vision
<code>data/textbook-excerpts.json</code>	Curriculum seed corpus (16 excerpts)
<code>data/demo-scenarios.json</code>	Three pre-loaded demo scenarios

Per-question latency budget

Today the end-to-end loop is roughly 7 seconds against live APIs. The breakdown: ~2.5s for the slowest of three parallel model calls, ~3s for the judge call, ~0.5s for retrieval and orchestration, ~1s for network. Headroom is the judge call — future work batches all judge queries into a single call to cut ~2s.

SECTION 10

UX flows

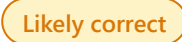
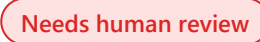
Flow 1 — Type-and-ask (the 80% case)

1. Child opens WhatsApp, taps Acharya, types a question in their language.
2. Typing dots appear in the assistant bubble (~7s).
3. Answer arrives with a green  badge in the bubble corner.
4. If the child wants the reasoning, they tap "Show consensus reasoning" in the chat header to expand a side panel showing all three model votes (in the local demo). On WhatsApp itself, this becomes a one-line link "Why this answer?" that opens a webview.

Flow 2 — Snap-the-page

1. Child taps the paperclip, picks a photo of the textbook page.
2. Image preview appears with "Reading question..." caption.
3. Within 2–3s, the extracted question text is auto-populated in the input field with the image preserved as an attachment.
4. Child taps Send (or edits the extracted text first if needed).
5. Same answer flow as Flow 1, with the image surfaced in the user message bubble.

Flow 3 — Disagreement

1. Question is sent normally.
2. Consensus engine runs; verdict comes back as  or .
3. Answer is delivered with the appropriate badge. For Likely correct, a soft caveat is appended ("Two of three models agreed on this; check the panel for the third opinion.").
4. For Needs review, no answer text is delivered; a message says "Acharya isn't confident enough to teach this answer. Your parent will see this in the dashboard." and the question goes to the parent dashboard's review queue.

Flow 4 — Parent dashboard

1. Parent opens the dashboard from the chat header menu (or a weekly WhatsApp link, in production).
2. Top of page: 4 summary cards, then the subject donut chart with legend.
3. Below: concept tracker with green/amber/red dots per topic.
4. Below: last 20 questions chronologically with their verdict icons, subject, language, and timestamp.
5. Tapping a Needs-review item opens the full question and the consensus panel for that exchange (P1).

SECTION 11

Privacy, security, & data handling

Children's data: the operating posture

Acharya's primary users are minors. The product's data posture must clear that bar before anything else.

Commitments (v1.0)

- **No training on child data.** Conversations are not used to fine-tune any model, nor sent to model providers under contracts that allow training. We use the Anthropic, OpenAI, and Google API endpoints whose default policy is no-training, and we do not opt into training programs.
- **No resale.** Conversation data is never sold, shared with advertisers, or used to build profiles for third parties.
- **Minimum retention.** The default retention is 90 days for the parent dashboard's recent-questions feature, then automatic purge. Verdicts and subject labels (without question text) may be retained longer for product analytics.
- **Parent visibility.** The parent always has access to the same data the system has — the dashboard is the primary interface for that visibility.
- **Indian data residency (target).** Production storage in an India-resident region of the chosen cloud provider, before public launch.

What we still need to do

- Privacy policy review by an Indian counsel before public launch.
- WhatsApp Business profile compliance with Meta's child-product rules.
- Decide opt-in vs opt-out semantics for the parent's weekly summary push.
- Decide whether voice notes are stored at all once transcribed.

Security baseline (v1.0)

- HTTPS only. Webhook signature verification for Meta callbacks.
- API keys in environment variables, never in source control. The MVP repo's `.gitignore` blocks `.env.local` at the root.
- Rate-limiting per phone number on the WhatsApp webhook to prevent abuse.
- Standard Postgres-side encryption at rest from the chosen managed provider.

SECTION 12

Success metrics

Engagement (per child)

- Daily active rate among onboarded children (target: $\geq 40\%$ by month 3 post-launch).
- Median questions per active day (target: 3–5).
- Median session length (target: 4–7 minutes — long enough to be useful, short enough to not be dependency).

Quality (per question)

- Distribution of verdicts: target $\geq 75\%$ Verified, $\leq 5\%$ Needs review.
- End-to-end latency p50 / p95 (target: $\leq 7s$ / $\leq 12s$).
- Per-model failure rate (target: $\leq 2\%$ per provider).

Trust (per parent)

- Weekly dashboard open rate among parents (target: $\geq 30\%$).
- Review-queue resolution rate (target: $\geq 60\%$ of Needs-review items reviewed within 7 days).

Cost

- Average per-question cost (target: $\leq ₹1.5$ at scale, including the judge call).

SECTION 13

Roadmap & milestones

Milestone	Target	Scope
M1 — MVP	Done	Local demo, 3-LLM consensus, vision intake, parent dashboard, three demo scenarios.
M2 — BSP onboarding	Q3 2026	Meta WhatsApp Business API approval via BSP. Single test number live.
M3 — Closed beta	Q4 2026	50 families across 2 cities. Voice intake. Persistence layer in production. Privacy policy filed.
M4 — Public launch	Q1 2027	Open WhatsApp signup. Weekly parent summary push. Tamil + Telugu added.
M5 — Teacher surface	Q3 2027	Teacher dashboard for classroom-wide question patterns and lesson preparation.

SECTION 14

Risks & open questions

Risks

Risk	Mitigation
Meta BSP onboarding takes >3 months and stalls launch.	The local demo de-risks investor and advisor conversations independently of the launch path. M2 timeline assumes worst-case.
Per-question cost stays above target at scale.	FR-3.7 caching and FR-3.8 adaptive sampling are the levers; both are P1/P2 because they only matter once volume is real.
Models converge on a wrong consensus (correlated failure mode).	The three vendors give us partial protection; for board-prep accuracy a future "expert review" mode could route flagged questions to subject-matter humans.
Parent doesn't engage with the dashboard.	The weekly WhatsApp push (FR-5.6) brings the dashboard to where the parent already is. Validate in M3 closed beta.
Vernacular models lag English meaningfully on niche topics.	Curriculum grounding (FR-4) is the primary mitigation. Verdict downgrades to Needs review when grounding fails.

Open questions

- Pricing model: parent-pays-monthly vs free-with-school-partnership vs freemium.
- Whether the consensus panel webview is needed on WhatsApp or whether a one-line caveat is enough for parents.
- How to detect and handle non-academic questions (a child asking a personal question, e.g. health or family) without rejecting them rudely.
- Whether voice replies (TTS) are worth the complexity in v1, or only voice-in.

SECTION 15

Glossary & appendix

Glossary

Term	Meaning in Acharya
Brooks–lyengar consensus	The fault-tolerant fusion algorithm at the heart of the product. See Section 8.
Verdict	One of Verified , Likely correct , Needs human review — computed per the threshold rules in Section 8.
Eligible model	A model whose subject matches the majority subject, whose surviving fact set is non-empty, and whose average fact-agreement score is ≥ 1 .

Surviving fact	A claim from a model whose fact-agreement score is ≥ 1 (at least one other model agreed).
Outlier fact	A claim with score 0 — no other model agreed. Discarded during fusion.
BSP	Business Solution Provider — a Meta-approved partner through which the WhatsApp Business API is accessed.

References

- Brooks, R. R., & Iyengar, S. S. (1996). *Robust Distributed Computing and Sensing Algorithm*. IEEE Computer.
- Anthropic Claude API documentation — vision & structured output.
- OpenAI Chat Completions API — JSON mode.
- Google Gemini API — `responseMimeType: application/json`.
- Meta WhatsApp Business API — webhook contract, message templates, BSP partner list.

— End of document —